

TL1

01_11.JSON出力

まとめ

日本工学院専門学校
デザインカレッジ

現状でもゲーム側でローダーを書けば実用レベルになっているはず。
先にローダーを書いてからレベルエディタの機能拡張をしていくとよいだろう。

ローダーについては特に理由がなければ外部のC++用JSONローダーを
組み込んで使うとよいだろう。

出力したファイルを解析してゲーム内でオブジェクトを並べる処理については
どちらにしろ自分で記述する必要がある。

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
1  {
2    "name": "scene",
3    "objects": [
4      {
5        "type": "LIGHT",
6        "name": "Light",
7        "transform": {
8          "translation": [
9            4.076245307922363,
10           1.0054539442062378,
11           5.903861999511719
12         ],
13         "rotation": [
14           37.261043548583984,
15           3.1637072563171387,
16           106.93630981445312
17         ],
18         "scaling": [
19           1.0,
20           0.9999999403953552,
21           1.0
22         ]
23       }
24     ],
25   }
```

入れ子構造を保ったまま、
各オブジェクトの詳細なプロパティも
記録されるようになった。

その他情報をパック

日本工学院専門学校
デザインカレッジ

つづき

```
#カスタムプロパティ 'file_name'
if "file_name" in object:
    json_object["file_name"] = object["file_name"]

#カスタムプロパティ 'collider'
if "collider" in object:
    collider = dict()
    collider["type"] = object["collider"]
    collider["center"] = object["collider_center"].to_list()
    collider["size"] = object["collider_size"].to_list()
    json_object["collider"] = collider
```

④

各カスタムプロパティのパック。

③ パックしたオブジェクトに1つでも [P5](#) の表にある以外の型のデータがあると、エンコードがエラーになってしまうので、このクラスに用意された `to_list()` を使ってlistに変換する。

今後他にもカスタムプロパティを増やしたら、このあたりにパック処理を追加するとよい。

その他情報をパック

日本工学院専門学校
デザインカレッジ

```
class MYADDON_OT_export_scene -parse_scene_recursive_json
```

```
#オブジェクトのローカルトランスフォームから  
#平行移動、回転、スケールを抽出
```

```
① trans, rot, scale = object.matrix_local.decompose()  
#回転を Quaternion から Euler (3軸での回転角) に変換  
rot = rot.to_euler()  
#ラジアンから度数法に変換
```

```
rot.x = math.degrees(rot.x)  
rot.y = math.degrees(rot.y)  
rot.z = math.degrees(rot.z)
```

```
#トランスフォーム情報をディクショナリに登録
```

```
② transform = dict()  
transform["translation"] = (trans.x, trans.y, trans.z)  
transform["rotation"] = (rot.x, rot.y, rot.z)  
transform["scaling"] = (scale.x, scale.y, scale.z)
```

```
#まとめて1個分のjsonオブジェクトに登録
```

```
③ json_object["transform"] = transform
```

つづく

P18 で、A「あとで処理を書く」
としていた「その他情報をパック」
する処理。

① トランスフォームの数値取得までは
非JSON書き出しの時と同じ処理。

② トランスフォーム情報の部分だけの
連想配列を作り、平行移動、回転、
スケーリングをそれぞれ書き込む。

③ まとめたトランスフォーム情報を、
"transform" というキーとペアにし
てオブジェクトに登録する。

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
1 {
2   "name": "scene",
3   "objects": [
4     {
5       "type": "LIGHT",
6       "name": "Light"
7     },
8     {
9       "type": "CAMERA",
10      "name": "Camera"
11     },
12     {
13       "type": "MESH",
14       "name": "ICO球AAA",
15       "children": [
16         {
17           "type": "MESH",
18           "name": "ICO球"
19         }
20       ]
21     },
22     {
23       "type": "EMPTY",
24       "name": "エンプティ",
25       "children": [
26         {
27           "type": "MESH",
28           "name": "ICO球B"
29         },
30         {
```

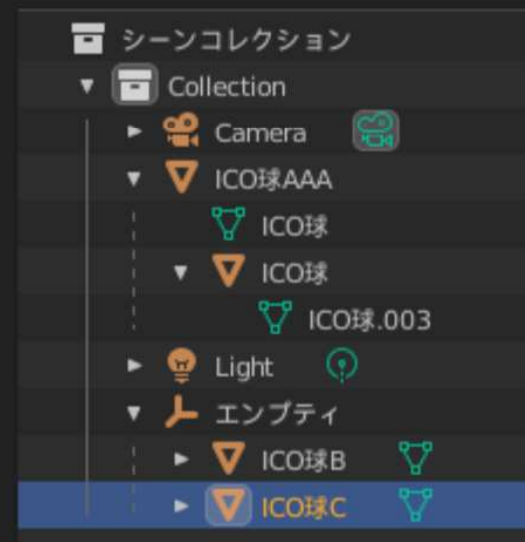
ICO球AAA

ICO球

再帰呼び出しによって、入れ子構造（ツリー構造）が実現した。

左の図だと“ICO球AAA”の子供が“ICO球”となっている。

その下では“エンプティ”の子供が“ICO球B”となっている。



子供リストの走査

日本工学院専門学校
デザインカレッジ

```
class MYADDON_OT_export_scene -parse_scene_recursive_json

#子ノードがあれば
if len(object.children) > 0:
    #子ノードリストを作成
    json_object["children"] = list()

    #子ノードへ進む(深さが1上がる)
    for child in object.children:
        self.parse_scene_recursive_json(json_object["children"], child, level + 1)
```

[P18](#) で、**B**「あとで処理を書く」としていた「直接の子供リストを走査」する処理。

Blenderオブジェクト1個分のパックオブジェクトに対して、
"children" というキーでリストを割り当てる。

自身と同じ関数を引数をずらして呼び出すことで、再帰関数として完成させる。

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
1  {
2    "name": "scene",
3    "objects": [
4      {
5        "type": "LIGHT",
6        "name": "Light"
7      },
8      {
9        "type": "CAMERA",
10       "name": "Camera"
11      },
12      {
13        "type": "MESH",
14        "name": "ICO球AAA"
15      },
16      {
17        "type": "EMPTY",
18        "name": "エンプティ"
19      }
20    ]
21 }
```

シーン直下のオブジェクトの

- 種類
- 名前

が表示されるようになった。

まだ再帰関数になっていないので、
第一世代（シーン直下）以外は表示されない。

シーン内の全オブジェクト走査

日本工学院専門学校
デザインカレッジ

```
class MYADDON_OT_export_scene -export_json
#シーン内の全オブジェクトについて
for object in bpy.context.scene.objects:

    #親オブジェクトがあるものはスキップ（代わりに親から呼び出すから）
    if (object.parent):
        continue

    #シーン直下のオブジェクトをルートノード(深さ0)とし、再帰関数で走査
    self.parse_scene_recursive_json(json_object_root["objects"], object, 0)
```

P13 で、**A**「あとで処理を書く」としていた「シーン内の全オブジェクト走査してパック」する処理。

シーンのオブジェクトリストを全走査して、1つのオブジェクトにパックする。
以前は「ファイルへの書き出し」だったものが「オブジェクトへのパック」に変わったただけなので、
構造は **非JSON書き出し** の時とほぼ同じ。

ツリー走査用の再帰関数

日本工学院専門学校
デザインカレッジ

```
level_editor.py -class MYADDON_OT_export_scene
```

```
def parse_scene_recursive_json(self,  
data_parent, object, level):
```

① #シーンのオブジェクト1個分のjsonオブジェクト生成
json_object = dict()
#オブジェクト種類
json_object["type"] = object.type
#オブジェクト名
json_object["name"] = object.name

A #Todo: その他情報をパック
あとでコードを書く

② #1個分のjsonオブジェクトを親オブジェクトに登録
data_parent.append(json_object)

B #Todo: 直接の子供リストを走査
あとでコードを書く

export_json() から呼び出す為の、
Blenderシーンにあるオブジェクト1個分について
処理する（パックする）再帰関数を作成。

[引数解説]

data_parent : 親となるlistを受け取る
object : 処理対象のBlenderオブジェクト
level : 再帰呼び出しの深さレベル

① Blenderオブジェクト1個分を連想配列(dict)
で持つことにした。

② 1個分の情報をまとめた後、親オブジェクト
の引数リストに子供として登録する

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
1  {
2    "name": "scene",
3    "objects": []
4  }
```

コンソール

```
レベルエディタが有効化されました。
シーン情報を Export します
{
  "name": "scene",
  "objects": []
}
シーン情報を Export しました
```

出力ファイルが整形されて見やすくなった。

この状態はまだ要素が少なくて分かりやすいが
この後情報が増えて複雑な入れ子構造になるので、
途中途中でファイル出力結果を確かめながら進めよう。

json.dumps()で整形

日本工学院専門学校
デザインカレッジ

```
#Todo: シーン内の全オブジェクト走査してパック  
あとでコードを書く
```

変更

```
#オブジェクトをJSON文字列にエンコード (改行・インデント付き)
```

```
json_text = json.dumps(json_object_root, ensure_ascii=False, cls=json.JSONEncoder, indent=4)
```

```
#コンソールに表示してみる
```

```
print(json_text)
```

非ASCII文字を
変換するか

エンコーダー

インデント幅

```
#ファイルをテキスト形式で書き出し用にオープン
```

```
#スコープを抜けると自動的にクローズされる
```

```
with open(self.filepath, "wt") as file:
```

```
#ファイルに文字列を書き込む
```

```
file.write(json_text)
```

エンコーダーを直接実行する代わりに json.dumps() を使うことで、
改行、インデントが入って整形されたJSON文字列を得られる。

実行結果

日本工学院専門学校
デザインカレッジ

出力したファイル

```
1 [{"name": "scene", "objects": []}]
```

この時点でのファイル出力を確認してみる。
（「MyMenu」→「シーン出力」）

上手くいっているのだが、改行がなくて見づらい。

関数呼び出し

日本工学院専門学校
デザインカレッジ

```
class MYADDON_OT_export_scene  
  
def execute(self, context):  
  
    print("シーン情報をExportします")  
    self.export_json()  
  
    self.report({'INFO'}, "シーン情報をExportしました")  
    print("シーン情報をExportしました")  
  
    return {'FINISHED'}
```

**execute() 内の export() を呼び出していた箇所を
export_json() の呼び出しに書き換える。**

エンコード・書き出し

日本工学院専門学校
デザインカレッジ

つづき

A

#Todo: シーン内の全オブジェクト走査してパック
あとでコードを書く

①

```
#オブジェクトをJSON文字列にエンコード
json_text = json.JSONEncoder().encode(json_object_root)
#コンソールに表示してみる
print(json_text)
```

```
#ファイルをテキスト形式で書き出し用にオープン
#スコープを抜けると自動的にクローズされる
with open(self.filepath, "wt", encoding="utf-8") as file:
```

②

```
#ファイルに文字列を書き込む
file.write(json_text)
```

① エンコーダーに、パックしたオブジェクトを渡してJSON文字列を得る。
(エンコード)

② ファイルを書き込み用にオープンし、JSON文字列を書き込む。

※日本語の書き出しは避けた方が無難だが、出力する場合はUTF-8に統一しておこう。

JSON版エクスポート

日本工学院専門学校
デザインカレッジ

```
level_editor.py -class MYADDON_OT_export_scene
```

```
def export_json(self):  
    """JSON形式でファイルに出力"""  
  
    #保存する情報をまとめるdict  
    json_object_root = dict()  
  
    #ノード名  
    json_object_root["name"] = "scene"  
    #オブジェクトリストを作成  
    json_object_root["objects"] = list()
```

つづく

エクスポートクラス(MYADDON_OT_export_scene)には既に export 関数があるが、

既存の非JSON版も機能として残す為、新たに json 用の export 関数を作成する。

json_object_root が、書き出すシーン情報をまとめておくためのオブジェクト。

トップを連想配列に決めたので、dict() であらたな連想配列オブジェクトを作成。

キー"name" のバリューを"scene"
キー"objects" のバリューをリストとして登録。

モジュールのインポート

日本工学院専門学校
デザインカレッジ

```
import json
```

スクリプトの先頭で、Python標準の json モジュールをインポートしておく。

これでJSON関連の機能が使えるようになる。

```
level_editor.py -class MYADDON_OT_export_scene
```

```
#オペレータ シーン出力
class MYADDON_OT_export_scene(bpy.types.Operator,
bpy_extras.io_utils.ExportHelper):
    bl_idname = "myaddon.myaddon_ot_export_scene"
    bl_label = "シーン出力"
    bl_description = "シーン情報をExportします"
    #出力するファイルの拡張子
    filename_ext = ".json"
```

変更

また、エクスポータークラスで
出力ファイルの拡張子設定を
".scene" から ".json" に変更しておく。

出力データをパックする

日本工学院専門学校
デザインカレッジ

現時点でのエクスポート（非JSON）は、

1. 書き込み用ファイルをオープン
2. オブジェクトリストを走査しながら各オブジェクトをファイルに書き出す

となっている。

しかし先ほども言ったように標準のJSONエンコーダーが
入れ子構造までまとめて処理してくれる関係上、
下のように処理順を整理した方が楽に処理できる。

1. オブジェクトリストを走査しながら、書き出す情報をパックする（まとめる）
2. まとめた情報をJSONエンコーダーに渡して、JSON文字列を得る
3. JSON文字列をファイルに書き出す

出力データをパックする

日本工学院専門学校
デザインカレッジ

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (real)	浮動小数点数
true	True
false	False
null	None

Python の json モジュールについては、Pythonの公式ページに [ドキュメント](#) がある。

[重要用語解説]

エンコード：PythonのデータをJSON文字列に変換すること

デコード：JSON文字列からPythonのデータに復元すること

json.JSONEncoder が標準のエンコーダーで、
左の図の様なルールでデータをJSON文字列に変換してくれる。

再帰処理で入れ子構造も含めて変換してくれるので、
標準のエンコーダーで十分使い物になる。

json.JSONEncoder を継承してクラスを作ることで、
エンコード機能の拡張も可能。

出力データをパックする

身近なJSONファイル

日本工学院専門学校
デザインカレッジ

```
1 {
2   "$schema": "vscode://schemas/color-theme",
3   "name": "Dark+ (default dark)",
4   "include": "./dark_vs.json",
5   "tokenColors": [
6     {
7       "name": "Function declarations",
8       "scope": [
9         "entity.name.function",
10        "support.function",
11        "support.constant.handlebars",
12        "source.powershell variable.other.member",
13        "entity.name.operator.custom-literal" // See https://er
14      ],
15      "settings": {
16        "foreground": "#DCDCAA"
17      }
18    },
19    {
20      "name": "Types declaration and references",
21      "scope": [
22        "meta.return-type",
23        "support.class",
24        "support.type",
25        "entity.name.type",
26        "entity.name.namespace",
27        "entity.other.attribute",
28        "entity.name.scope-resolution",
29        "entity.name.class",
30        "storage.type.numeric.go",
31        "storage.type.byte.go",
32        "storage.type.boolean.go",
33        "storage.type.string.go",
34        "storage.type.uintptr.go",
```

JSONは色々なところで使われている。

例えばVSCoideの配色テーマもJSONだ。

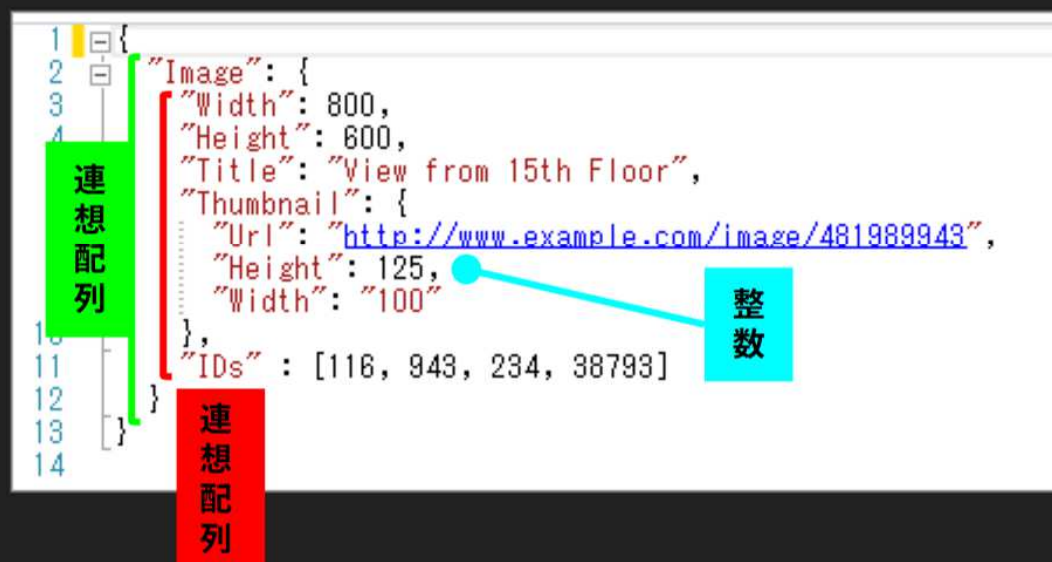
[VSCoideのインストールパス

¥resources¥app¥extensions¥theme-defaults¥them
es¥dark_plus.json]

をVisual Studio や VSCoideなどの
テキストエディタで開くと、左の様に
JSON形式で保存されているのが分かる。

JSONの入れ子構造

日本工学院専門学校
デザインカレッジ



ルート連想配列の "Image" をキーとする要素の
バリューがまた連想配列で、その連想配列の
"Thumbnail" をキーとする要素のバリューが
また連想配列で
その連想配列のキー "Height" のバリューが整数125

JSONは入れ子の構造が可能なので複雑なデータ
も格納できる。

例えば配列の中に配列を持ったり（ジャグ配
列、二次元配列）

配列の中に連想配列を持ったり
連想配列のデータ部に文字列を配列を混ぜて格
納したり

かなり融通が効く。

逆に言えば自分のデータをどのように整理して
格納するか
仕様を決めて扱う必要がある。

JSONのデータ種類

日本工学院専門学校
デザインカレッジ

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (real)	浮動小数点数
true	True
false	False
null	None

JSONは

連想配列、配列、文字列、整数、浮動小数点数、ブーリアン、ヌルのデータを持つことができる。

そして、Python の json モジュールを使うと、左図のようなルールで相互に型変換できる。

Python 以外にも様々な言語にライブラリがあり、ルールもそこまで複雑ではないのでJSONの書式に沿って自分で「読み込み／書き込み」処理を記述することも十分可能。

[C++のjsonライブラリ nlohmann-json](#)

JSONとは

日本工学院専門学校
デザインカレッジ

JSONファイルの例

```
1 {  
2   "name": "KogakuinText",  
3   "type": "dark",  
4   "colors": {  
5     "scrollbarSlider.background": "■" "#00000050",  
6     "scrollbarSlider.hoverBackground": "■" "#00000075",  
7     "scrollbarSlider.activeBackground": "■" "#61616170",  
8     "sideBar.background": "■" "#212121",  
9     "sideBar.foreground": "■" "#eeeeee",  
10    "sideBarTitle.foreground": "■" "#FFFFFF",  
11    "list.hoverForeground": "■" "#80D8FF",  
12    "list.hoverBackground": "■" "#303030",  
13    "list.inactiveSelectionBackground": "■" "#ffffff10",  
14    "tab.activeBackground": "■" "#212121",  
15    "tab.inactiveForeground": "■" "#848484",  
16    "tab.inactiveBackground": "■" "#212121",  
17    "tab.unfocusedActiveForeground": "■" "#EEEEFF",  
18    "activityBar.background": "■" "#212121",  
19    "editor.background": "■" "#212121",  
20    "editor.foreground": "■" "#eeffff",  
21    "list.inactiveSelectionBackground": "■" "#00000050"
```

JSONとは

「JavaScript Object Notation」の略で、「JavaScriptのオブジェクトの書き方を元にしたデータ定義方法」のこと。

テキストベースのデータフォーマットで、全部のデータを1つの文字列に格納できる。

類似の書式にXMLがある。

JSONとは

今回の目標

日本工学院専門学校
デザインカレッジ

- レベルデータファイルをJSON形式での書き出しに変更する
- レベルエディタ完成！

TL1

01_11.JSON出力